

Build Tools

Storefront Next uses [Vite](#) as its build tool, providing fast development builds through native ES modules and optimized production builds through Rollup. The toolchain includes native TypeScript support and Vitest for testing.

Vite Configuration

The `vite.config.ts` file configures these plugins for your storefront.

- `reactRouter()` : Enables React Router 7 framework mode with file-based routing and Server-Side Rendering (SSR).
- `tailwindcss()` : Compiles Tailwind CSS styles.
- `tsconfigPaths()` : Resolves TypeScript path aliases like `@/components`.
- `storefrontNextPlugin()` : Prepares your build for deployment to Managed Runtime (MRT), including bundle formatting, manifest generation, and React Router compatibility patches.

You can add more [Vite plugins](#) to customize your build.

```
1 // vite.config.ts
2 import { defineConfig } from "vite";
3 import { reactRouter } from "@react-router/dev/vite";
4 import tailwindcss from "@tailwindcss/vite";
5 import tsconfigPaths from "vite-tsconfig-paths";
6 import storefrontNextPlugin from "@salesforce/storefront-next";
7
8 export default defineConfig({
9   plugins: [reactRouter(), tailwindcss(), tsconfigPaths(), storefrontNextPlugin()],
10 });
```

Storefront Next Plugin Options

The `storefrontNextPlugin` accepts this optional configuration object.

```
1 storefrontNextPlugin({
2   readableChunkNames: true,
3 });
```

Contents

Vite Configuration

[React Router Configuration](#)[TypeScript Configuration](#)[Testing with Vitest](#)[Development Commands](#)[Source Maps](#)[Bundle Size Monitoring](#)



readable
chunk file
names for
easier
debugging.
Useful with
bundle
analyzer.

When `readableChunkNames` is enabled, chunk files are named based on their source location.

```
</> 1 // With readableChunkNames: true
2 (package)-(react-router)-(dist)-chunk.a1b2c3.js
3 (components)-(ui)-(button)-index.d4e5f6.js
4
5 // Default (readableChunkNames: false)
6 chunk.a1b2c3.js
7 index.d4e5f6.js
```

Vite Plugins

Plugin	Purpose
<code>@react-router/dev/vite</code>	React Router 7 framework mode integration
<code>@tailwindcss/vite</code>	Tailwind CSS compilation
<code>vite-tsconfig-paths</code>	TypeScript path alias resolution
<code>storefrontNextPlugin</code>	MRT deployment and bundle optimization
<code>vite-plugin-bundleSize</code>	Bundle size validation
<code>rollup-plugin-visualizer</code>	Bundle analysis visualization

React Router Configuration

The `react-router.config.ts` file uses a preset that enforces standard configuration for B2C Commerce.

```
</> 1 // react-router.config.ts
2 import type { Config } from '@react-router/dev/config'
3 import { odysseyPreset } from '@salesforce/storefront-
4
5 export default {
```



- `appDirectory: './src'` – App source directory
- `buildDirectory: 'build'` – Build output directory
- `serverModuleFormat: 'cjs'` – Server bundle format for MRT compatibility
- `ssr: true` – Server-side rendering enabled
- Middleware support enabled

TypeScript Configuration

Storefront Next uses TypeScript with strict settings enabled by default.

```
1 {
2   "compilerOptions": {
3     "target": "ES2022",
4     "module": "ES2022",
5     "moduleResolution": "bundler",
6     "jsx": "react-jsx",
7     "strict": true,
8     "noUnusedLocals": true,
9     "noUnusedParameters": true,
10    "paths": {
11      "@/*": [".src/*"],
12      "@config/server": [".config.server.ts"]
13    }
14  }
15 }
```

The `@/*` path alias lets you import from the `src` directory using absolute paths:

```
1 import { Button } from "@components/ui/button";
2 import { useCart } from "@hooks/use-cart";
```

Testing with Vitest

Storefront Next uses [Vitest](#) for unit testing. Vitest integrates natively with Vite, sharing the same configuration and transformation pipeline.

Configure testing options in the `test` section of `vite.config.ts`.

```
1 // vite.config.ts
2 export default defineConfig({
3   // ... other config
4   test: {
5     globals: true,
6     environment: "jsdom",
7     setupFiles: [".vitest.setup.ts"],
8     include: ["**/*.test.ts", "**/*.spec.ts", "**/*.tsx"],
```

[Try for free](#)

```
14 },  
15 };
```

Development Commands

Command	Description
<code>pnpm dev</code>	Start development server with hot module replacement.
<code>pnpm build</code>	Create a production build.
<code>pnpm start</code>	Preview a production build locally.
<code>pnpm push</code>	Deploy to MRT.
<code>pnpm typecheck</code>	Run TypeScript type checking.
<code>pnpm test</code>	Run Vitest tests with coverage.
<code>pnpm bundleSize:test</code>	Build and validate bundle sizes.
<code>pnpm bundleSize:analyze</code>	Build and visualize bundle composition.

Source Maps

Production builds generate source maps by default. Source maps map minified code back to your original source files, giving you readable stack traces when errors occur.

Configure Source Maps in Vite

Control source map generation in `vite.config.ts` using Vite's `[build.sourcemap]` (<https://vite.dev/config/build-options.html#build-sourcemap>) option.

```
</> |    
  
1 // vite.config.ts  
2 export default defineConfig({  
3   build: {  
4     sourcemap: true, // default  
5   },  
6   plugins: [reactRouter(), tailwindcss(), tsconfigPath  
7 ]});
```



<code>"hidden"</code>	Generates <code>.map</code> files but omits the <code>sourceMappingURL</code> comment in bundles.
-----------------------	---

<code>false</code>	Disables source map generation entirely.
--------------------	--

Enable Source Maps on Managed Runtime

Source maps are included in your deployed bundle, but MRT must be configured to use them at runtime. When enabled, MRT starts your server with the `--enable-source-maps` Node.js flag, which gives you original file names and line numbers in error stack traces.

Using Runtime Admin

1. Log in to [Runtime Admin](#).
2. Click your project.
3. Click your target environment.
4. Click **Environment Settings**.
5. In the Advanced section, click **Edit**.
6. Enable **Source Maps**.
7. Click **Update** and wait for the bundle to redeploy.

Using the Managed Runtime API

```
1 curl -X PATCH \
2 https://cloud.mobify.com/api/projects/{project_id}/tar
3 --header "Authorization: Bearer $API_KEY" \
4 --header 'Content-Type: application/json' \
5 --data '{"enable_source_maps": true}'
```

Bundle Size Monitoring

Storefront Next enforces bundle size limits to maintain performance. Configure limits in `package.json`.

```
1 "bundlesize": {
2   "server": [
3     { "name": "**/*", "limit": "5 MB" }
4   ],
5   "client": [
6     { "name": "assets/-root.*.js", "limit": "113 k
7     { "name": "**/*", "limit": "50 kB" }
8   ]
9 }
```

Run `pnpm bundlesize:test` to validate that your build stays within limits. If a chunk exceeds the baseline 50-KB limit, add an

[Try for free](#)

DID THIS ARTICLE SOLVE YOUR ISSUE?

Let us know so we can improve!

[Share your feedback](#)

DEVELOPER CENTERS

[Heroku](#)
[MuleSoft](#)
[Tableau](#)
[Commerce Cloud](#)
[Lightning Design System](#)
[Einstein](#)
[Quip](#)

POPULAR RESOURCES

[Documentation](#)
[Component Library](#)
[APIs](#)
[Trailhead](#)
[Sample Apps](#)
[AppExchange](#)

COMMUNITY

[Trailblazer Community](#)
[Events and Calendar](#)
[Partner Community](#)
[Blog](#)
[Salesforce Admins](#)
[Salesforce Architects](#)

© Copyright 2026 Salesforce, Inc. [All rights reserved](#). Various trademarks held by their respective owners. Salesforce, Inc. Salesforce Tower, 415 Mission Street, 3rd Floor, San Francisco, CA 94105, United States

[Legal](#) [Terms of Service](#) [Privacy Information](#) [Responsible Disclosure](#) [Trust](#) [Contact](#) [Use of Cookies](#)

[Cookie Preferences](#)  [Your Privacy Choices](#)

